"An Introduction to Prolog"
Part I

Viviana Mascardi
viviana.mascardi@unige.it

University of Genoa – DIBRIS

Dibris

Disclaimer

- This presentation may contain material protected by copyright laws.
- In particular, these lessons are based on the material that Peter Schachte made available on the Web years ago, as part of the Logic and Computation Course at the Computer Science Department of the University Melbourne
- We made this material available on the Web only to ensure timely dissemination among our students, and is meant only for students' personal use.
- Any use different from the students' personal use is prohibited.
- V. Mascardi thanks Professor Schachte for the courtesy of sharing his teaching material on Prolog.

Dibris

1. Predicates
2. Making Queries
3. Variables
   - Multiple Directions
4. Compound Queries
   - More Predicates
   - Constraints
5. Clause Bodies
   - Extending the Example
6. Recursive Predicates
   - Querying Recursive Predicates
7. Disjunction
   - Variables
8. Compound Terms
   - Compound Terms
9. Variables in Terms
   - Unification
10. Recursive Structures
   - Programming Numbers as Terms

Predicates

In most programming languages, the basic abstraction is the *function* or perhaps *method*
In Prolog, the basic abstraction is the predicate or relation

A function is a mapping from some inputs to some outputs

A predicate instead indicates whether or not some entities are related in a particular way

In logic programming, may test whether some entities are related in a particular way, or to find entities that are related to other entities

Dibris

Predicates

Here is a simple Prolog program giving information about the British royal family:

```
parent(elizabeth, charles).
parent(diana, william).
parent(diana, harry).
parent(philip, charles).
parent(charles, william).
parent(charles, harry).
```

Here `parent` is a predicate
Each line above is a separate clause
Each clause is a separate true statement: should be able to understand each clause separately
A Prolog program may consist of many files, each defining many predicates, each defined by many clauses

Dibris

Making Queries

We can use this program to ask about family relationships

Most Prolog systems, such as the SICStus Prolog and TuProlog systems we will be using, provide an interactive environment

Allows loading and testing code

Prolog prompt is ?-

Load source file by wrapping file name in single quotes and square brackets, followed by full stop

Usual extension for Prolog source code is `.pl`

Pose queries by typing them at the prompt; Prolog will answer them with `Yes` or `No`

Making Queries

For these lectures, boxed text like this represents terminal sessions

```
C:\ java alice.tuprologx.ide.CUIConsole
tuProlog 1.3.0  - DEIS,  Universita' di Bologna a Cesena
Sat Oct 14 11:02:46 CEST 2006

?- ['royals.pl'].
% royals.pl compiled 0.00 sec, 1,200 bytes
Yes
?- parent(diana, harry).
Yes
?- parent(elizabeth, diana).
No
```

Variables

We can query of whom Elizabeth is parent using a variable:

```
?- parent(elizabeth, X).
X = charles
Yes
```

Prolog variables begin with a *capital letter* or an *underscore*, and follow with letters, digits, and underscores

N.B. Prolog variables are very different than variables in other programming languages

Like Haskell: you cannot change the value of a variable once it is set

Read query as: "find `X` such that `elizabeth` is parent of `X`"

Dibris

Variables

Some queries have more than one answer:

```
?- parent(diana, X).
X = william ;
X = harry ;
No
```

When Prolog prints an answer, it waits to see if you want more answers

Hit semicolon to ask for more answers; hit return if no more answers are needed

After asking for another answer after `harry`, Prolog says `No`, meaning there are no more answers

Multiple Directions

We could run this query the other way around:

```
?- parent(X, william).
X = diana ;
X = charles ;
No
```

"Find `X` such that `X` is parent of `william`"

This is possible because our program is *declarative*: it specifies information without specifying how it must be used

Well-written logic programs exhibit this flexibility

This is not always practical, as we will see later

Multiple Directions

In general, for *each* argument of a predicate we may pass a variable or a value Either way, Prolog finds all matching solutions

Think of the values specified as constraining the solutions, while variables do not impose a constraint

A query with only variables imposes no constraints:

```
?- parent(X, Y).
X = elizabeth
Y = charles ;
X = diana
Y = william
```

Compound Queries

Prolog queries can invoke multiple predicates, separating them with commas

If variables appear more than once, all occurrences must have same value (as in Algebra)

```
?- parent(elizabeth,X), parent(X,Y).
X = charles
Y = william ;
X = charles
Y = harry ;
No
```

Compound Queries

Our program so far includes no information about gender; add it now:

```
female(elizabeth).
female(diana).
male(philip).
male(charles).
male(william).
male(harry).
```

`male` and `female` are predicates defining the gender of the royals

Predicates can take any number of arguments

The number of arguments is called the `arity`

A predicate with only one argument essentially defines a property that certain entities may have

Motherhood

Give a Prolog query that determines the mother of `william`

Motherhood

Give a Prolog query that determines the mother of `william`

```
?- parent(X, william), female(X).
```

Motherhood

Give a Prolog query that determines the mother of `william`

```
?- parent(X, william), female(X).
```

Answer (equally good):

```
?- female(X), parent(X, william).
```

Constraints

A good way to think of compound queries is that each part of the query imposes a constraint on the solution

Prolog will only show solutions that satisfy all the constraints

Some constraints have no solutions

In general, constraints can be placed in any order

Some orders may be more efficient than others

Some orders may not work as expected; more later

Constraints

```
?- parent(X,Y), parent(Y,Z), male(Z), female(X).
X = elizabeth
Y = charles
Z = william ;
X = elizabeth
Y = charles
Z = harry ;
No
?- parent(X,Y), parent(Y,X).
No
?- male(X), female(X).
No
```

Constraints

Predicates can be defined in terms of other predicates

A clause may have two parts: a [of clause]head and a [of clause]body

Head and body are separated by :- (read it as "if")

Head looks like a predicate with arguments, some of which may be variables

Body looks like a query

Means head is true if body is true

Omit :- if body is empty (as in previous examples)

Clause with no body is called a unit *clause* or a *fact*

Dibris

Extending the Example

We can define a grandparent as a parent's parent:

```
grandparent(G, C) :-
parent(G, P),
parent(P, C).
```

Read: "G is grandparent of C if G is parent of P and P is parent of C"
Can use this to find grandparents or grandchildren:

```
?- grandparent(elizabeth, X).
X = william ;
X = harry ;
No
?- grandparent(X, william).
X = elizabeth ;
X = philip ;
No
```

Dibris

Family Relationships

1. Define a predicate `father(Dad,Kid)` that holds when `Dad` is father of `Kid`
2. Define a predicate `greatgrandparent(Adult,Child)` that holds when `Adult` is a great grandparent of `Child`

Recursive Predicates

Predicates can also be defined recursively:

```
ancestor(A, A).
ancestor(A, D) :-
parent(A, C),
ancestor(C, D).
```

The first clause says that everyone is their own ancestor — not the usual definition, but common in computer science

The second clause says that the parents of your ancestors are your ancestors

This code can be used both for finding ancestors and descendants

Dibris

Querying Recursive Predicates

```
?- ancestor(A,harry).
A = harry ;
A = elizabeth ;
A = diana ;
A = philip ;
A = charles ;
No
?- ancestor(elizabeth, D).
D = elizabeth ;
D = charles ;
D = william ;
D = harry ;
No
```

Disjunction

Disjunction in Prolog uses the ; operator, e.g.:

```
person(X) :-
( good(X) ; bad(X) ).
```

Could also have been written:

```
person(X) :- good(X).
person(X) :- bad(X).
```

Variables

How can a *variable* be a data structure?

A Prolog variable is just *a term whose value you don't know yet*

Don't *assign* to variables, *constrain* them
A variable can be bound to any term — even another variable
Can't change value of variable once bound
More like a variable in algebra than a conventional programming language

Compound Terms

The Compound term is *Prolog's sole data structuring abstraction*

A compound term has:

- a **functor**, which is an atom
- one or more **arguments**, which can be any terms

Syntax: functor first, then arguments in parentheses, separated by commas
Compound terms look just like predicate invocations

The number of arguments is called the *arity*

No need to declare it — just use it

Dibris

Compound Terms

Use compound terms where you would use (a pointer to) a struct in C.

Use functor to indicate the kind (type?) of term.

No need to allocate a structure and assign to members: just write it down.

E.g., to create a term holding an (X,Y) position of (20,10):

```
position(20,10)
```

To create an employee data structure:

```
employee(1234, 'Jones', 'James', 100000)
```

Variables in Terms

E.g., to create a term holding an (X,Y) position of (`X`,`Y`) where `X` and `Y` are variables:

```
position(X,Y)
```

To create an employee data structure:

```
employee(Num, Surname, Firstname, Salary)
```

Valid whether or not values of variables are determined

Anonymous Variables

In Prolog, = is *used for both giving values to variables and comparing terms for equality*

```
?- 4 = 4.

Yes
?- 4 = 5.

No
?- X = 5.

X = 5 ;

No
```

```
?- 5 = V.

V = 5 ;

No
?- X = Y.

X = _G168
Y = _G168 ;

No
```

N.B. Variables appearing in different queries are unrelated.

Anonymous Variables

= performs *unification*, tries to *make two terms same by binding variables appearing in the terms*

Unification is a powerful sort of pattern matching

```
?- foo(A,2,C,D) = foo(1,X,Y,X).
A = 1
C = _G302
D = 2
X = 2
Y = _G302 ;
No
?- bar(X, X) = bar(1, 2).
No
```

Prolog also uses unification for argument passing

Unification - Exercises

We haven't formally defined unification yet. Nevertheless, try to find bindings for these variables to make these pairs of terms the same.

1. `f(X, 17) = f(Y, Y)`

2. `f(42, U, a) = f(V, f(V,W), W)`
   NB: `f` is used with both arity 2 and 3; no problem

Unification - Exercises

We haven't formally defined unification yet. Nevertheless, try to find bindings for these variables to make these pairs of terms the same.

1. `f(X, 17) = f(Y, Y)`

   `X = 17, Y = 17`

2. `f(42, U, a) = f(V, f(V,W), W)`
   NB: `f` is used with both arity 2 and 3; no problem

Unification - Exercises

We haven't formally defined unification yet. Nevertheless, try to find bindings for these variables to make these pairs of terms the same.

1. `f(X, 17) = f(Y, Y)`

   `X = 17, Y = 17`

2. `f(42, U, a) = f(V, f(V,W), W)`
   NB: `f` is used with both arity 2 and 3; no problem

   `V = 42, W = a, U = f(42,a)`

Recursive Structures

Any argument of any compound term can be any other term — including a term with the same functor

This is how Prolog implements recursive structures, such as lists, trees, etc.

One **excellent** strategy for writing predicates that process recursive structures:

1. write predicate that recognizes recursive data structure
2. use this definition as a "skeleton" for other predicates, adding extra arguments and goals as needed

Programming Numbers as Terms

For example, we could represent a natural number as either `0` or a term `s(X)` (1 more than `X`) where `X` represents a natural number
NB: This is not really how Prolog does maths

```
nat(0).                  % 0 is a natural number
nat(s(X)) :- nat(X).     % s(X) is a nat if X is

add(0, Y, Y).            % adding 0 to Y gives Y
add(s(X), Y, s(Z)) :-    % adding X+1 to Y gives Z+1
        add(X, Y, Z).    %         where Z is X + Y
```

Note that structure of `add/3` follows that of `nat/1`
(We refer to predicates as *name*/*arity* since Prolog allows different predicates with same name and different arity)

Example Queries

```
?- nat(s(s(0))).
Yes

?- nat(X).
X = 0 ;
X = s(0) ;
X = s(s(0))
Yes

?- add(s(s(0)), X, Y).
X = _G254
Y = s(s(_G254)) ;
No
```

```
?- add(s(s(0)), s(s(s(0))), X).
X = s(s(s(s(s(0))))) ;
No

?- add(s(s(0)), X, s(s(s(s(s(0)))))).
X = s(s(s(0))) ;
No

?- add(X, s(s(s(0))), s(s(s(s(s(0)))))).
X = s(s(0)) ;
No

?- add(X,Y,s(s(0))).
X = 0, Y = s(s(0)) ;
X = s(0), Y = s(0) ;
X = s(s(0)), Y = 0 ;
No
```

Exercise

Define a predicate `mult(X,Y,Z)` such that `X` times `Y` equals `Z`. Use the add/3 predicate defined above.

```
mult(0, _, 0).              %multiplying anything by 0 gives 0
mult(s(X), Y, Z) :-          %Z = Y(X+1) if
    mult(X, Y, W),           %W = XY
    add(W, Y, Z).            %and Z = W + Y
```